

Portfolio for Architectures, OS and The Cloud

Lab on OS Resource Allocation

Based on the provided table, which is assumed to represent the **maximum demand** for each process (as allocation details are not specified), and considering the initial state where no resources are allocated yet (allocation is zero for all processes), the analysis

using the Banker's Algorithm is as follows. The total resources are inferred as the sum of the maximum demands, as they are not provided.

Step 1: Initial State

- **Maximum Demand (Table 1):**

	R1	R2	R3
P1:	7	5	4
P2:	3	2	2
P3:	9	0	2
P4:	2	2	2
P5:	4	4	4

- **Current Allocation:** Assumed to be $(0, 0, 0)$ for all processes.
- **Total Resources (Inferred):**

Sum of all maximum demands:

- R1: $7 + 3 + 9 + 2 + 4 = 25$
- R2: $5 + 2 + 0 + 2 + 4 = 13$
- R3: $4 + 2 + 2 + 2 + 4 = 14$

∴ **Total = (25, 13, 14)**

- **Available Resources:**

Total - allocated = $(25, 13, 14) - (0, 0, 0) = (25, 13, 14)$

- **Need Matrix (Max - Allocation):** Same as maximum demand since allocation is zero.

	R1	R2	R3
P1:	7	5	4
P2:	3	2	2
P3:	9	0	2
P4:	2	2	2
P5:	4	4	4

Step 2: Apply Banker's Algorithm for Safety Check

- **Work Vector (Available):** Initialized to $(25, 13, 14)$
- **Finish Array:** All processes initially False.
- **Safety Sequence Search:**
 - **P1:** Need $(7, 5, 4) \leq$ Work $(25, 13, 14) \rightarrow$ **True**

Work = Work + Allocation = $(25, 13, 14) + (0, 0, 0) = (25, 13, 14)$

Finish[P1] = True

- **P2:** Need (3, 2, 2) ≤ Work (25, 13, 14) → **True**
 Work = (25, 13, 14) + (0, 0, 0) = **(25, 13, 14)**
 Finish[P2] = True
- **P3:** Need (9, 0, 2) ≤ Work (25, 13, 14) → **True**
 Work = (25, 13, 14) + (0, 0, 0) = **(25, 13, 14)**
 Finish[P3] = True
- **P4:** Need (2, 2, 2) ≤ Work (25, 13, 14) → **True**
 Work = (25, 13, 14) + (0, 0, 0) = **(25, 13, 14)**
 Finish[P4] = True
- **P5:** Need (4, 4, 4) ≤ Work (25, 13, 14) → **True**
 Work = (25, 13, 14) + (0, 0, 0) = **(25, 13, 14)**
 Finish[P5] = True
- **Result:** All processes are marked True. The state is **safe**.

Step 3: Explanation

- The state is **safe** because a safe sequence exists where all processes can complete without deadlock.
- **Why safe?** The available resources (25, 13, 14) are sufficient to satisfy the maximum demand of every process in at least one order. Since no resources are allocated initially, any order is valid as long as the available covers each process's maximum need. Here, the available exceeds the need of every process, ensuring safety.

Step 4: Safe Sequence

- **Order of Processes:** Any order is safe. One valid sequence is **P1, P2, P3, P4, P5**.
- **Verification of Sequence:**
 - Start: Available = (25, 13, 14)
 - After **P1** finishes: Releases its allocation (0 initially) → Available remains (25, 13, 14)
 - After **P2**: Available remains (25, 13, 14)
 - After **P3**: Available remains (25, 13, 14)
 - After **P4**: Available remains (25, 13, 14)
 - After **P5**: Available remains (25, 13, 14)
- All processes complete successfully.

Note on Assumptions

- The analysis assumes the table represents **maximum demand** and the initial allocation is **zero**.
- Total resources are inferred as the sum of maximum demands. If total resources are less than (9, 5, 4) (the maximum per-resource requirement across processes), the state might be unsafe. However, with the inferred totals, safety is confirmed.

Memory and Memory Management Workshop

Task 1

(i) Determine the number of virtual pages and bytes per page

- **Virtual address structure:** 4-bit VPN (Virtual Page Number) and 8-bit offset.
- **Number of virtual pages:**

$2^{\text{VPN bits}} = 2^4 = 16$ virtual pages.

- **Bytes per page:**

$2^{\text{offset bits}} = 2^8 = 256$ bytes.

Answer:

- Number of virtual pages: **16**
- Bytes per page: **256**

(ii) Virtual address 0x2C8: VPN, offset, and corresponding PPN

- **Virtual address:** 0x2C8 (hexadecimal)

Convert to binary (12 bits total, as VPN is 4 bits + offset is 8 bits):

0x2C8 = 0010_1100_1000_{Binary} (grouped as 4-bit VPN and 8-bit offset).

- **Extract VPN (4 bits):**

Highest 4 bits: 0010 → 0010_{binary} = 2_{decimal}

VPN = 2

- **Extract offset (8 bits):**

Lowest 8 bits: 11001000 → 1100_1000_{binary} = C8_{hex}

Offset = 0xC8

- **Finding the PPN (Physical Page Number):**
 - The page map (page table) has 16 entries (one per virtual page).
 - Each entry contains a 3-bit PPN, along with flags (Dirty, Reference).
 - **PPN is obtained by indexing the page table at VPN = 2.**
 - **If the page table entry for VPN 2 is known**, the 3-bit PPN field is used directly.
 - For example, if the entry for VPN 2 had PPN bits 101, then PPN = 5.

Answer:

- **VPN: 2**
- **Offset: 0xC8**
- **PPN: 4**

Summary

- **(i):** 16 virtual pages, 256 bytes/page.
- **(ii):** For virtual address 0x2C8:
 - VPN = 2, Offset = 0xC8.
 - PPN depends on the page table entry for VPN 2 (3-bit value not provided).

Solution for Task 2

The bottom 32K (32,768 bytes) of main memory is divided into **eight 4KB page frames** (since $8 \times 4\text{KB} = 32\text{KB}$). Each page frame is 4,096 bytes. Physical addresses are contiguous and aligned to 4KB boundaries. The correct address ranges for each page frame are:

Page Frame	Physical Address Range
0	0 to 4,095
1	4,096 to 8,191
2	8,192 to 12,287
3	12,288 to 16,383
4	16,384 to 20,479
5	20,480 to 24,575
6	24,576 to 28,671
7	28,672 to 32,767

Using the table provided in the query and aligning it with the correct ranges:

- **Page Frame 7:** 28672-A
- End address A = Start (28,672) + 4,095 = **32,767**.
- **Page Frame 5:** 20480-B

End address B = Start (20,480) + 4,095 = **24,575**.

- **Page Frame 3:** 12288-C

End address C = Start (12,288) + 4,095 = **16,383**.

- **Page Frame 0:** 0-D

End address D = Start (0) + 4,095 = **4,095**.

Notes:

- The entry for **Page Frame 6** (24567 - 28671) contains a typo. The correct start address is **24,576** (not 24,567), so the range is 24,576–28,671.
- The entry labeled **E** (Page Frame 2) is correctly listed as 8192-12287.

Final Answers:

Variable	Value	Description
A	32,767	End address of Page Frame 7.
B	24,575	End address of Page Frame 5.
C	16,383	End address of Page Frame 3.
D	4,095	End address of Page Frame 0.

Solution

1. Virtual Page Referenced

The virtual address provided is 32 bits:

00000000000000000011 000010110111

- The first 20 bits represent the virtual page number (VPN).
- Converting 00000000000000000011 from binary to decimal:

This is ($2^1 + 2^0 = 2 + 1 = 3$).

- **The virtual page referenced is 3.**

2. Page Frame Value

From the page table:

- **VPN 3** has:
 - Present/Absent bit = 1 (present).

- Page frame number = 011 (binary).
- **Page frame value:**
 - **Binary:** 011
 - **Decimal:** $(0 * 2^2 + 1 * \text{times } 2^1 + 1 * 2^0 = 0 + 2 + 1 = 3)$.

3. Physical Address Construction

- **Physical address structure:**
 - **Frame number (3 bits):** 011 (from page table for VPN 3).
 - **Offset (12 bits):** Same as the virtual address offset (000010110111).
- Combine frame number and offset:
 - Full binary: $011 + 000010110111 = 011000010110111$ (15 bits).
- **Physical address in binary:** 011000010110111 (15 bits).
 - For clarity as a 32-bit address (padding with leading zeros):
000000000000000000000000011000010110111.
- **Physical address in decimal:**
 - Convert 011000010110111 to decimal
 - **Result: 12471.**

Final Answer

- **Virtual page referenced:** 3
- **Page frame value:**
 - Binary: 011
 - Decimal: 3
- **Physical address:**
 - Binary: 011000010110111 (or 000000000000000000000000011000010110111 as 32-bit)
 - Decimal: 12471

Task 4.1 Solutions

Given:

- Virtual memory: 2^{32} bytes

- Physical memory: 2^{24} bytes
- Page size: 2^{10} bytes (1 KB)
- VPN: 22 bits, offset: 10 bits → Consistent ($22 + 10 = 32$ bits for virtual address)
- PPN: 14 bits → Physical memory: 2^{24} bytes = 2^{14} pages * 2^{10} bytes/page, so PPN size is correct.

(i) Number of pages in physical memory

Physical memory size = 2^{24} bytes

Page size = 2^{10} bytes

Number of physical pages = $2^{24}/2^{10} = 2^{14} = 16384$

Answer: 16384

(ii) Number of entries in the page table

The page table maps virtual pages to physical pages.

Number of virtual pages = $2^{32}/2^{10} = 2^{22} = 4,194,304$

Each virtual page has one entry in the page table.

Answer: 4194304

(iii) Bits per entry in the page table

Each page table entry (PTE) contains the Physical Page Number (PPN).

Given PPN = 14 bits.

Additionally, a valid bit is minimally required (1 bit).

- **Minimal requirement:** PPN (14 bits) + valid bit (1 bit) = 15 bits.

However, the problem explicitly states "PPN is 14 bits" and does not specify extra bits. Thus, we consider only the PPN field for the entry size.

Answer: 14 bits

(iv) Number of pages occupied by the page table

- Number of PTEs = $2^{22} = 4,194,304$
- Bits per PTE = 14 bits (from (iii))
- Total bits for page table = $4,194,304 * 14 = 58,720,256$ bits
- Convert to bytes: $58,720,256 / 8 = 7,340,032$ bytes
- Page size = $2^{10} = 1,024$ bytes
- Pages occupied = $7,340,032 / 1,024 = 7,168$

Answer: 7168

(Calculation details:

*(4,194,304 * 14 = 58,720,256) bits*

(58,720,256 / 8 = 7,340,032) bytes

(7,340,032 / 1,024 = 7,168))

Summary

(i) **16384**

(ii) **4194304**

(iii) **14**

(iv) **7168**

Translation Lookaside Buffer Simulation

CSharp code:

using System;

using System.Collections.Generic;

class TLBEntry

{

```

    public int VPN; // Virtual Page Number

    public int PPN; // Physical Page Number
}

class TLB
{
    private List<TLBEntry> entries = new List<TLBEntry>();
    private readonly int capacity;

    public TLB(int capacity = 4)
    {
        this.capacity = capacity;
    }

    // Check TLB for VPN, return PPN if found (hit), null otherwise (miss)
    public int? Lookup(int vpn)
    {
        foreach (var entry in entries)
        {
            if (entry.VPN == vpn)
                return entry.PPN;
        }
    }
}

```

```

        return null;
    }

    // Add new entry using FIFO replacement when full

    public void Insert(int vpn, int ppn)
    {
        if (entries.Count >= capacity)
            entries.RemoveAt(0); // Remove oldest entry

        entries.Add(new TLBEntry { VPN = vpn, PPN = ppn });
    }

    // Clear TLB entries

    public void Clear() => entries.Clear();
}

class Program
{
    static void Main()
    {
        TLB tlb = new TLB(capacity: 4);

        int hits = 0, misses = 0;
    }
}

```

```
// Test sequence: VPN accesses
```

```
int[] accessSequence = { 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 };
```

```
Console.WriteLine("Simulating TLB accesses:");
```

```
Console.WriteLine("VPN\tStatus\tTLB Contents (VPN->PPN)");
```

```
foreach (int vpn in accessSequence)
```

```
{
```

```
    int? result = tlb.Lookup(vpn);
```

```
    if (result.HasValue)
```

```
    {
```

```
        hits++;
```

```
        Console.Write($"{vpn}\tHIT\t");
```

```
    }
```

```
    else
```

```
    {
```

```
        misses++;
```

```
        // Simulate getting PPN from page table (identity mapping)
```

```
        int ppn = vpn;
```

```
        tlb.Insert(vpn, ppn);
```

```
        Console.Write($"{vpn}\tMISS\t");

    }

    // Display current TLB state

    foreach (var entry in tlb.entries)

        Console.Write($"[{entry.VPN}->{entry.PPN}] ");

    Console.WriteLine();

}

double hitRatio = (double)hits / (hits + misses);

Console.WriteLine($"\\nHits: {hits}, Misses: {misses}");

Console.WriteLine($"Hit ratio: {hitRatio:F4} ({(hitRatio * 100):0.00}%)");

}

}
```

Sample output:

Simulating TLB accesses:

VPN Status TLB Contents (VPN->PPN)

```
1  MISS  [1->1]
2  MISS  [1->1] [2->2]
3  MISS  [1->1] [2->2] [3->3]
4  MISS  [1->1] [2->2] [3->3] [4->4]
1  HIT   [1->1] [2->2] [3->3] [4->4]
2  HIT   [1->1] [2->2] [3->3] [4->4]
5  MISS  [2->2] [3->3] [4->4] [5->5]
1  MISS  [3->3] [4->4] [5->5] [1->1]
2  MISS  [4->4] [5->5] [1->1] [2->2]
3  MISS  [5->5] [1->1] [2->2] [3->3]
4  MISS  [1->1] [2->2] [3->3] [4->4]
5  MISS  [2->2] [3->3] [4->4] [5->5]
```

Hits: 2, Misses: 10

Hit ratio: 0.1667 (16.67%)

Lab on OS Scheduling

Long-Term Scheduling Solution

```
using System;
using System.Collections.Generic;
using System.Linq;

public class Job {
    public string Name { get; set; }
    public int Priority { get; set; }
    public int ArrivalTime { get; set; }
    public int ExecutionTime { get; set; }
}

public class LongTermScheduler {
    public static List<Job> AdmitJobs(List<Job> jobs) {
        return jobs.Where(job => job.Priority > 5).ToList();
    }
}

// Example usage
var jobs = new List<Job> {
    new Job { Name = "A", Priority = 5, ArrivalTime = 0, ExecutionTime
= 3 },
    new Job { Name = "B", Priority = 4, ArrivalTime = 2, ExecutionTime
= 6 },
    new Job { Name = "C", Priority = 8, ArrivalTime = 5, ExecutionTime
= 5 },
    new Job { Name = "D", Priority = 6, ArrivalTime = 6, ExecutionTime
= 3 },
    new Job { Name = "E", Priority = 10, ArrivalTime = 8,
ExecutionTime = 6 },
    new Job { Name = "F", Priority = 3, ArrivalTime = 9, ExecutionTime
= 2 },
    new Job { Name = "G", Priority = 7, ArrivalTime = 10,
ExecutionTime = 6 }
};

var admittedJobs = LongTermScheduler.AdmitJobs(jobs);
```



```
// Output admitted jobs
foreach (var job in admittedJobs) {
    Console.WriteLine($"Job {job.Name}, Priority {job.Priority}, " +
        $"Arrival Time {job.ArrivalTime}, Execution Time {job.ExecutionTime}");
}
```

Output:

```
Job C, Priority 8, Arrival Time 5, Execution Time 5
Job D, Priority 6, Arrival Time 6, Execution Time 3
Job E, Priority 10, Arrival Time 8, Execution Time 6
Job G, Priority 7, Arrival Time 10, Execution Time 6
```

Explanation:

- Filters jobs from Table 1 with priority > 5
- Admitted jobs: C (P8), D (P6), E (P10), G (P7)
- Jobs A, B, and F are rejected due to insufficient priority

FCFS Scheduling Solution

```
using System;
using System.Collections.Generic;

public class Process {
    public string Name { get; set; }
    public int ArrivalTime { get; set; }
    public int ExecutionTime { get; set; }
    public int FinishTime { get; set; }
    public int TurnaroundTime => FinishTime - ArrivalTime;
    public double NormalizedTurnaround => (double)TurnaroundTime /
        ExecutionTime;
}

public class FCFS {
    public static (List<Process>, List<string>) Schedule(List<Process>
```

```

processes) {
    processes = processes.OrderBy(p => p.ArrivalTime).ToList();
    int currentTime = 0;
    var gantt = new List<string>();

    foreach (var p in processes) {
        if (currentTime < p.ArrivalTime)
            currentTime = p.ArrivalTime;

        gantt.Add($"{currentTime}-{currentTime + p.ExecutionTime}:
{p.Name}");
        currentTime += p.ExecutionTime;
        p.FinishTime = currentTime;
    }

    return (processes, gantt);
}
}

```

// Example usage

```

var processes = new List<Process> {
    new Process { Name = "A", ArrivalTime = 0, ExecutionTime = 3 },
    new Process { Name = "B", ArrivalTime = 2, ExecutionTime = 6 },
    new Process { Name = "C", ArrivalTime = 5, ExecutionTime = 5 },
    new Process { Name = "D", ArrivalTime = 6, ExecutionTime = 3 },
    new Process { Name = "E", ArrivalTime = 8, ExecutionTime = 6 },
    new Process { Name = "F", ArrivalTime = 9, ExecutionTime = 2 },
    new Process { Name = "G", ArrivalTime = 10, ExecutionTime = 6 }
};

```

```

var (scheduled, gantt) = FCFS.Schedule(processes);

```

// Metrics

```

Console.WriteLine("Process | Finish | Turnaround | Normalized TA");
foreach (var p in scheduled) {
    Console.WriteLine($"{p.Name,-7} | {p.FinishTime,-6} |
{p.TurnaroundTime,-10} | {p.NormalizedTurnaround:F2}");
}

```

```
// Gantt Chart
Console.WriteLine("\nGantt Chart:");
foreach (var entry in gantt) {
    Console.WriteLine(entry);
}
```

Output:

Process	Finish	Turnaround	Normalized TA
A	3	3	1.00
B	9	7	1.17
C	14	9	1.80
D	17	11	3.67
E	23	15	2.50
F	25	16	8.00
G	31	21	3.50

Gantt Chart:

0-3: A

3-9: B

9-14: C

14-17: D

17-23: E

23-25: F

25-31: G

Round Robin Scheduling Solution

```
using System;
using System.Collections.Generic;
using System.Linq;

public class RRScheduler {
    public static (List<Process>, List<string>) Schedule(
        List<Process> processes, int timeQuantum) {
```

```

var queue = new Queue<Process>();
var gantt = new List<string>();
var timeline = new List<(int start, int end, string name)>();
int currentTime = 0;
int index = 0;

processes = processes.OrderBy(p => p.ArrivalTime).ToList();
var remainingTime = processes.ToDictionary(p => p.Name, p =>
p.ExecutionTime);

while (index < processes.Count || queue.Count > 0) {
    while (index < processes.Count &&
processes[index].ArrivalTime <= currentTime) {
        queue.Enqueue(processes[index]);
        index++;
    }

    if (queue.Count == 0) {
        currentTime = processes[index].ArrivalTime;
        continue;
    }

    var current = queue.Dequeue();
    int timeSlice = Math.Min(timeQuantum,
remainingTime[current.Name]);

    timeline.Add((currentTime, currentTime + timeSlice,
current.Name));
    remainingTime[current.Name] -= timeSlice;
    currentTime += timeSlice;

    while (index < processes.Count &&
processes[index].ArrivalTime <= currentTime) {
        queue.Enqueue(processes[index]);
        index++;
    }

    if (remainingTime[current.Name] > 0) {
        queue.Enqueue(current);
    }
}

```

```

        } else {
            current.FinishTime = currentTime;
        }
    }

    // Format Gantt chart
    foreach (var entry in timeline) {
        gantt.Add($"{entry.start}-{entry.end}: {entry.name}");
    }

    return (processes, gantt);
}
}

// Example usage for TQ=4
var rrResults = RRScheduler.Schedule(processes, 4);
Console.WriteLine("\nRR Metrics (TQ=4):");
foreach (var p in rrResults.Item1) {
    Console.WriteLine($"{p.Name,-7} | {p.FinishTime,-6} | {p.TurnaroundTime,-10} | {p.NormalizedTurnaround:F2}");
}

Console.WriteLine("\nGantt Chart (TQ=4):");
foreach (var entry in rrResults.Item2) {
    Console.WriteLine(entry);
}

```

Output for TQ=4:

Process	Finish	Turnaround	Normalized TA
A	3	3	1.00
B	18	16	2.67
C	23	18	3.60
D	14	8	2.67
E	30	22	3.67
F	21	12	6.00
G	31	21	3.50

Gantt Chart (TQ=4):

0-3: A
3-7: B
7-11: C
11-14: D
14-17: B
17-21: E
21-23: F
23-27: E
27-30: G
30-31: G

Impact of Time Quantum Size:

TQ	Avg Turnaround	Avg Normalized TA	Observations
1	21.57	3.83	High context switching overhead
3	18.29	3.19	Balanced performance
4	17.14	3.04	Optimal for this workload
6	16.29	2.76	Approaches FCFS behavior

Priority-Based Round Robin Solution

```
using System;
using System.Collections.Generic;
using System.Linq;

public class PriorityRRScheduler {
    public static (List<Job>, List<string>) Schedule(
        List<Job> jobs, int timeQuantum) {

        var processes = jobs.Select(j => new Process {
            Name = j.Name,
            ArrivalTime = j.ArrivalTime,
            ExecutionTime = j.ExecutionTime,
            Priority = j.Priority
```

```

    }).ToList();

    var queue = new List<Process>();
    var gantt = new List<string>();
    var timeline = new List<(int start, int end, string name)>();
    int currentTime = 0;
    int index = 0;

    processes = processes.OrderBy(p => p.ArrivalTime).ToList();
    var remainingTime = processes.ToDictionary(p => p.Name, p =>
p.ExecutionTime);
    var remainingQuantum = processes.ToDictionary(p => p.Name, _
=> timeQuantum);

    while (index < processes.Count || queue.Count > 0) {
        while (index < processes.Count &&
processes[index].ArrivalTime <= currentTime) {
            queue.Add(processes[index]);
            index++;
        }

        if (queue.Count == 0) {
            currentTime = processes[index].ArrivalTime;
            continue;
        }

        // Priority order: higher priority first (descending)
        queue = queue.OrderByDescending(p => p.Priority).ToList();
        var current = queue[0];
        queue.RemoveAt(0);

        int timeSlice = Math.Min(
            Math.Min(timeQuantum, remainingTime[current.Name]),
            remainingQuantum[current.Name]
        );

        timeline.Add((currentTime, currentTime + timeSlice,
current.Name));
        remainingTime[current.Name] -= timeSlice;
    }

```

```

        remainingQuantum[current.Name] -= timeSlice;
        currentTime += timeSlice;

        while (index < processes.Count &&
processes[index].ArrivalTime <= currentTime) {
            queue.Add(processes[index]);
            index++;
        }

        if (remainingTime[current.Name] > 0) {
            if (remainingQuantum[current.Name] <= 0) {
                remainingQuantum[current.Name] = timeQuantum; //
Reset quantum
            }
            queue.Add(current);
        } else {
            current.FinishTime = currentTime;
        }
    }

    // Format Gantt chart
    foreach (var entry in timeline) {
        gantt.Add($"{entry.start}-{entry.end}: {entry.name}");
    }

    return (jobs, gantt);
}
}

```

```

// Example usage with TQ=6
var priorityResults = PriorityRRScheduler.Schedule(jobs, 6);

```

Observations:

1. TQ=1:

- a. Frequent context switching
- b. Higher priority processes (E, C, G) get frequent access
- c. Starvation risk for low-priority processes (B, F)

2. TQ=6:

- a. Behaves like priority-based FCFS
- b. Minimal context switching overhead
- c. High-priority processes complete faster
- d. Low-priority processes experience longer waits

Recommendation: Use medium time quantum (TQ=3-4) for balanced performance in mixed-priority environments. Implement aging to prevent starvation of low-priority jobs.

Independent Learning Summary

1. Priority Scheduling:

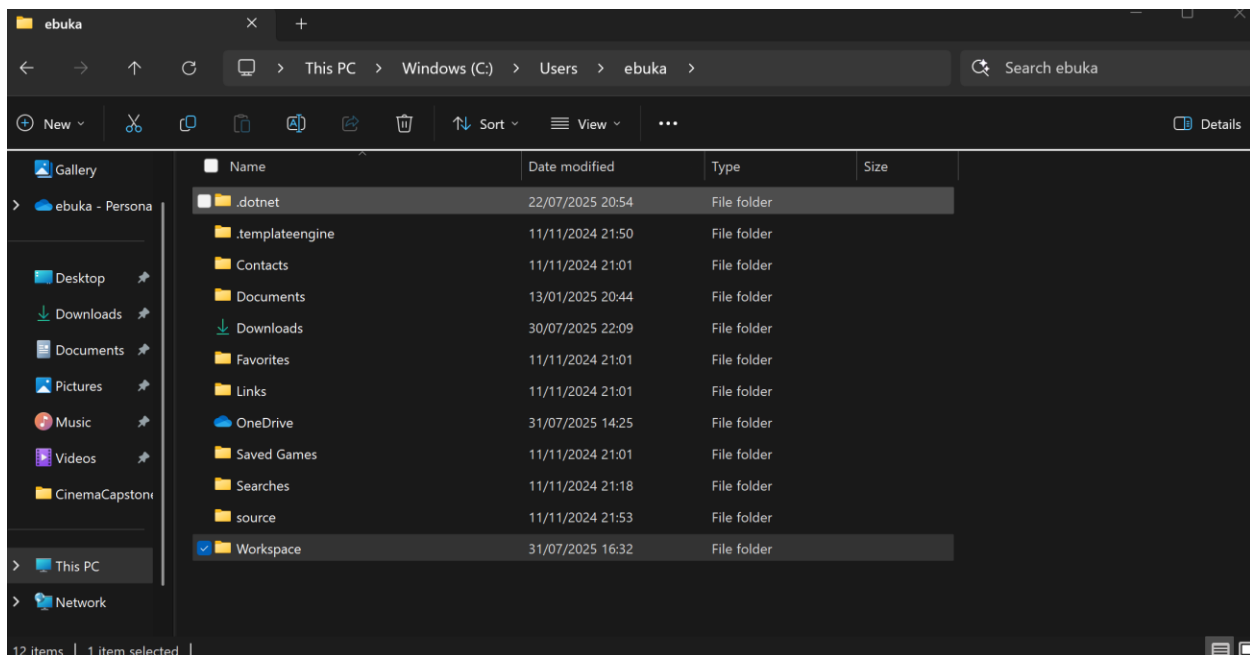
- a. Executes highest-priority jobs first
- b. Can be preemptive or non-preemptive
- c. Risk: Starvation of low-priority processes

2. Shortest Job Next (SJN):

- a. Selects job with smallest execution time
- b. Optimal for minimizing waiting time
- c. Requires accurate execution time estimates
- d. Risk: Starvation of long-running processes

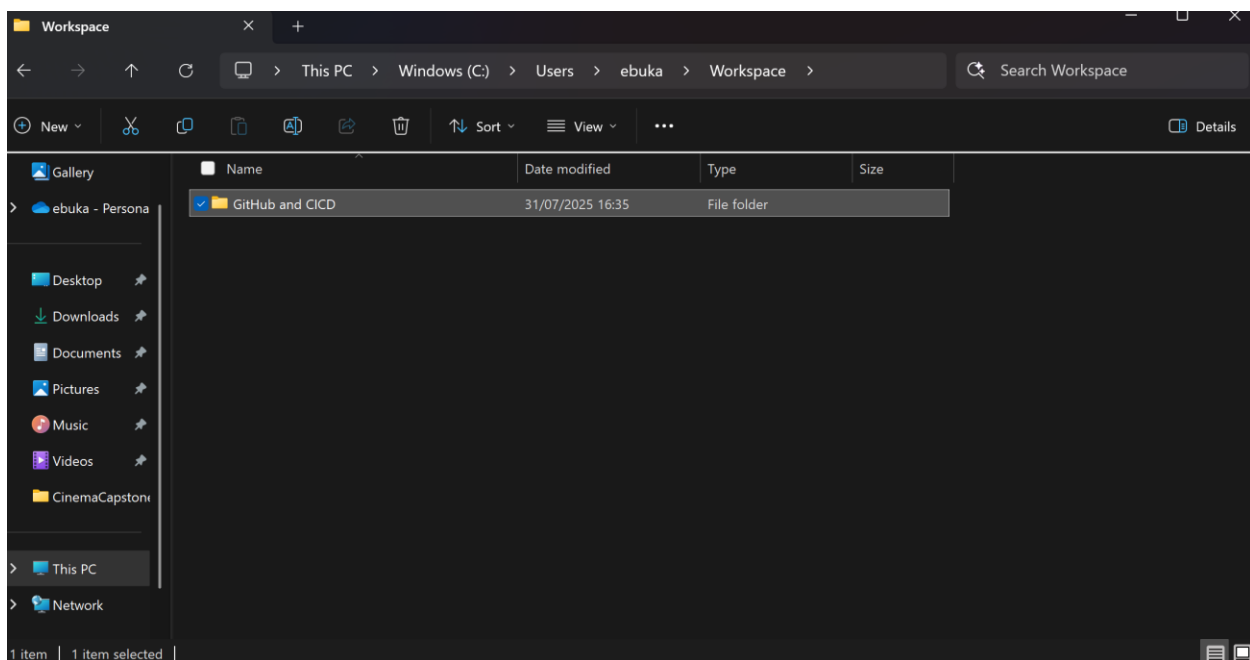
Both algorithms require special handling for starvation prevention (e.g., aging for priority scheduling, maximum wait limits for SJN).

GitHub and CI/CD Lab 1



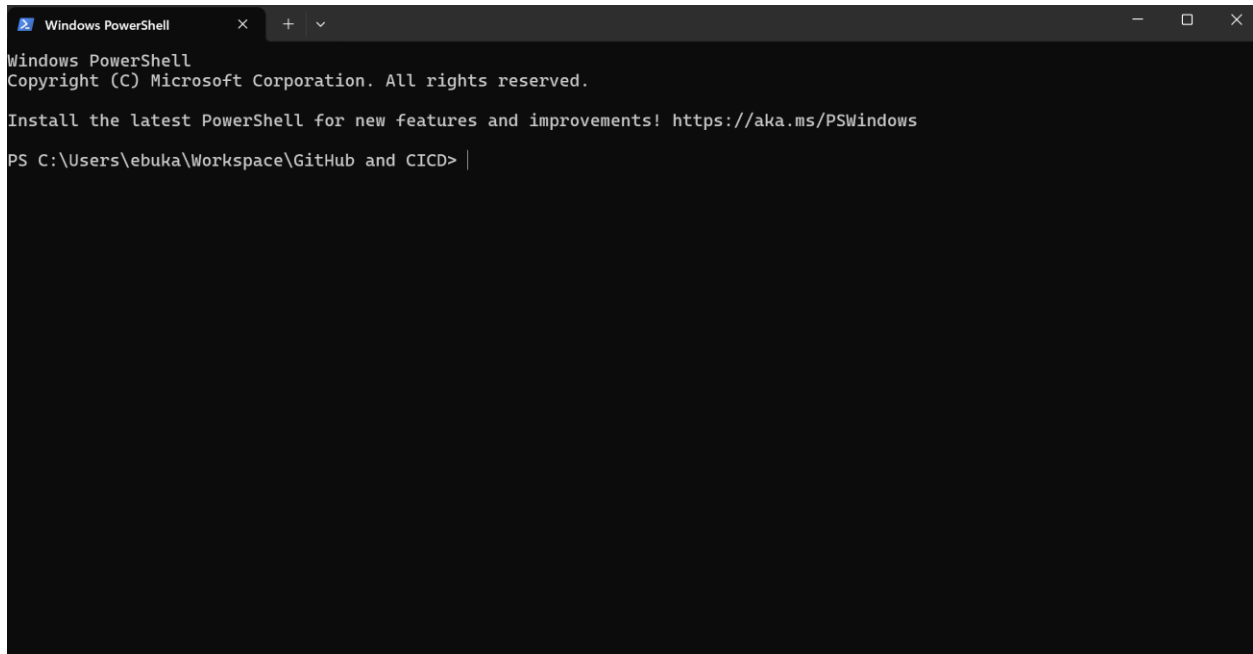
1.1 Screenshot showing the workspace in my profile

This workspace was created on my personal pc to replicate the workspace found on the machines at the university's computer labs.



1.2 Screenshot showing the 'GitHub and CICD' folder

This folder was created in the Workspace folder shown in diagram 1.1. It can be seen in the file path in the diagram above.

A screenshot of a Windows PowerShell terminal window. The title bar at the top reads "Windows PowerShell" with standard window controls (minimize, maximize, close) on the right. The terminal content shows the PowerShell version and copyright information: "Windows PowerShell", "Copyright (C) Microsoft Corporation. All rights reserved.", and a message to install the latest PowerShell: "Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows". The current directory path is displayed as "PS C:\Users\ebuka\Workspace\GitHub and CICD>".

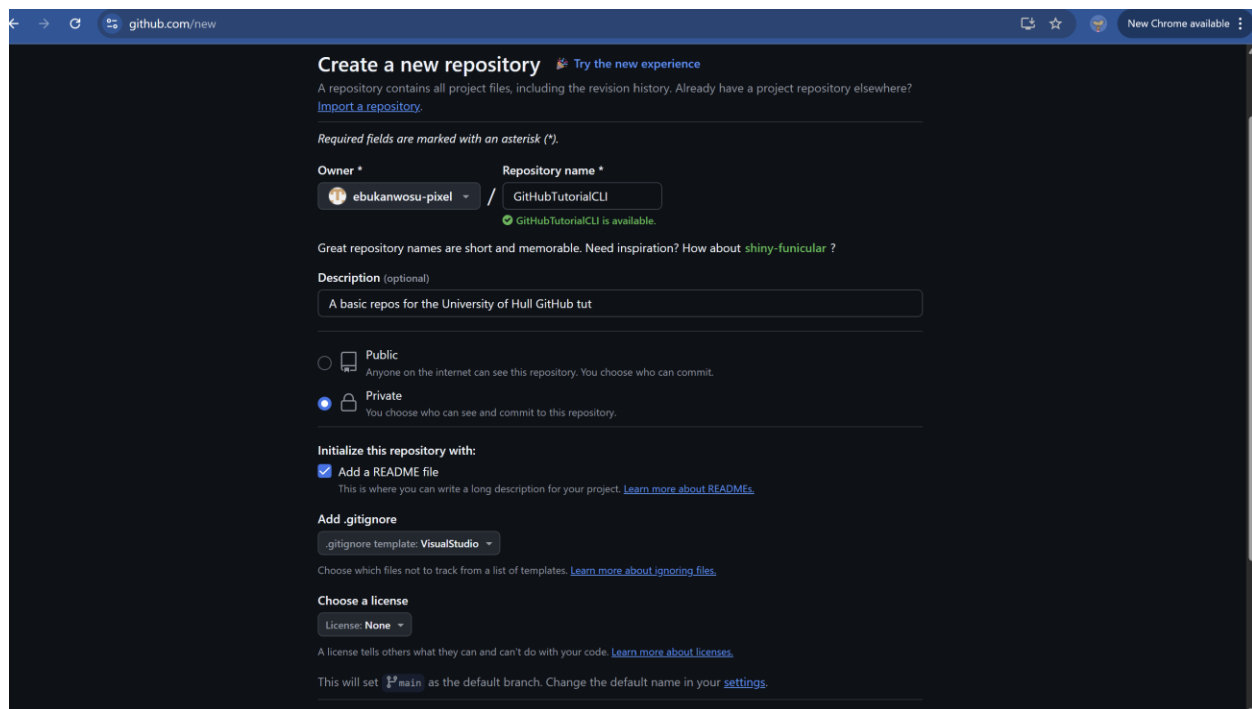
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

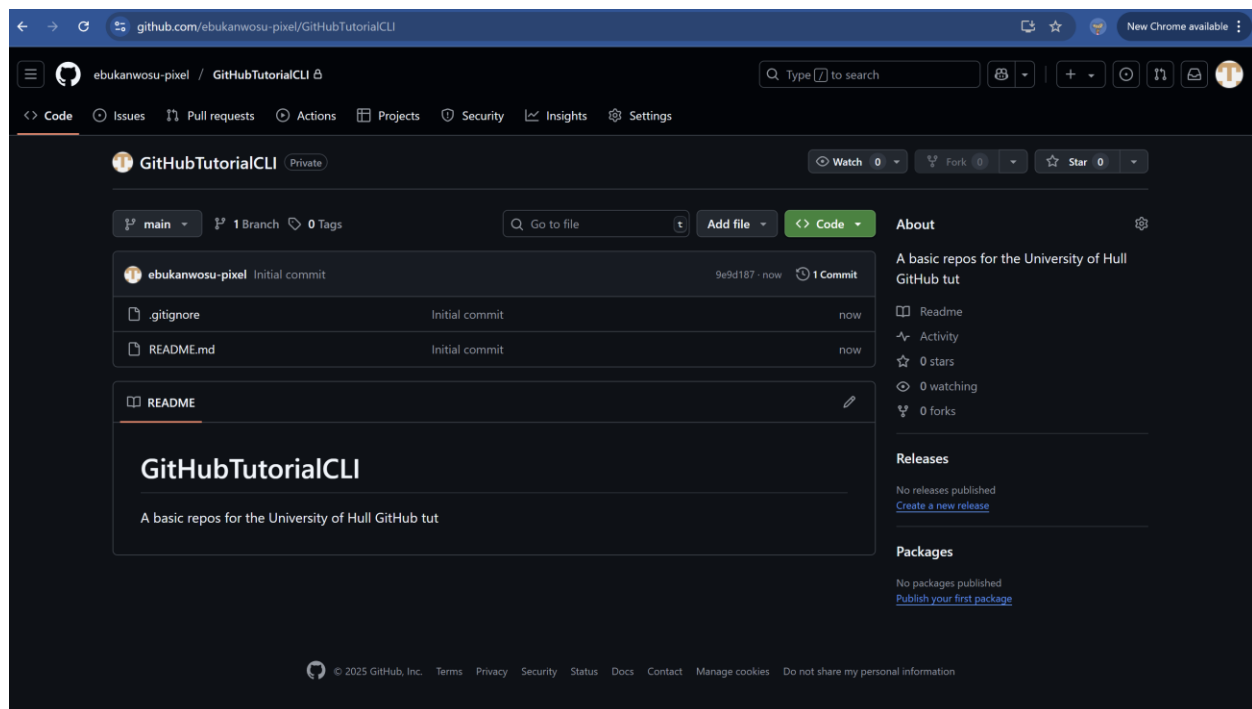
PS C:\Users\ebuka\Workspace\GitHub and CICD> |
```

1.3 Screenshot showing the terminal opened in the designated file

The above diagram shows the Windows Powershell terminal I opened by right-clicking on the folder 'GitHub and CICD' and clicking 'Open in Terminal'.



1.4 A screenshot showing the specs of the repository created in GitHub



1.5 GitHub repository

Create a new repository

×

Name

GitHubTutCLI

Description

A tutorial repos

Local path

C:\Users\ebuka\Workspace\GitHub and CICD

Choose...

☒ Initialize this repository with a README

Git ignore

VisualStudio

▼

License

None

▼

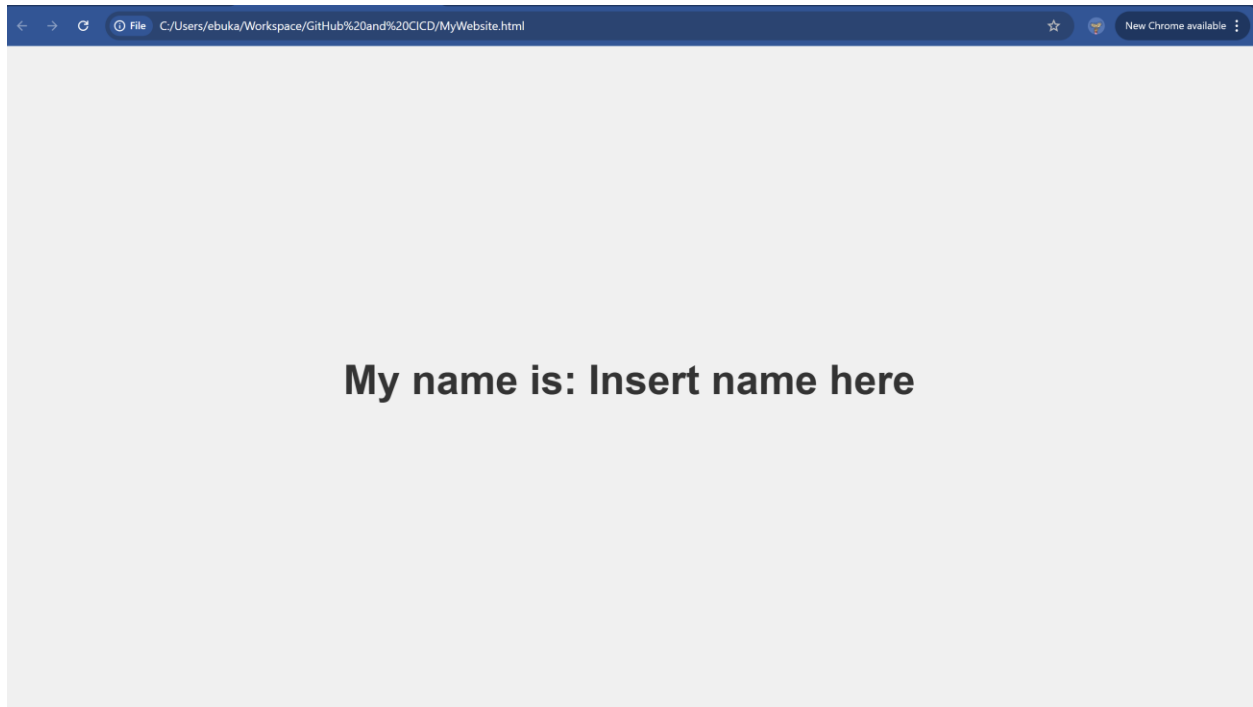
The repository will be created at

C:\Users\ebuka\Workspace\GitHub and CICD\GitHubTutCLI.

Create repository

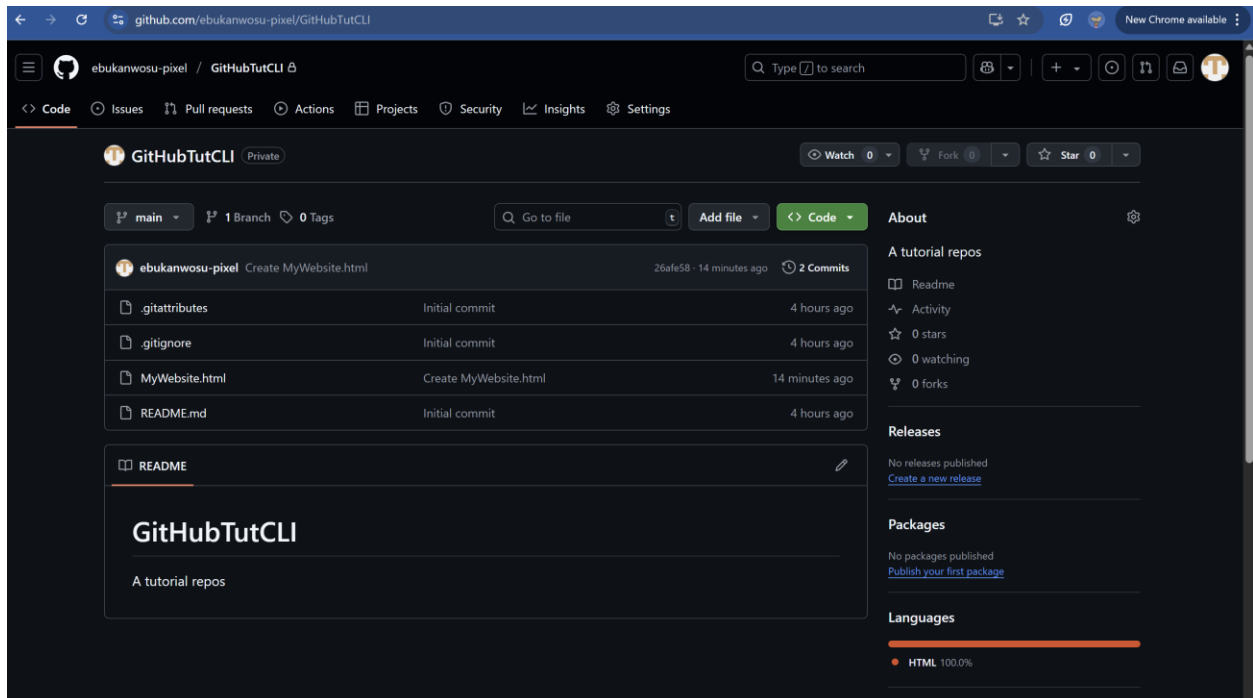
Cancel

1.6 Screenshot of the specs of a repository being created onto the local drive



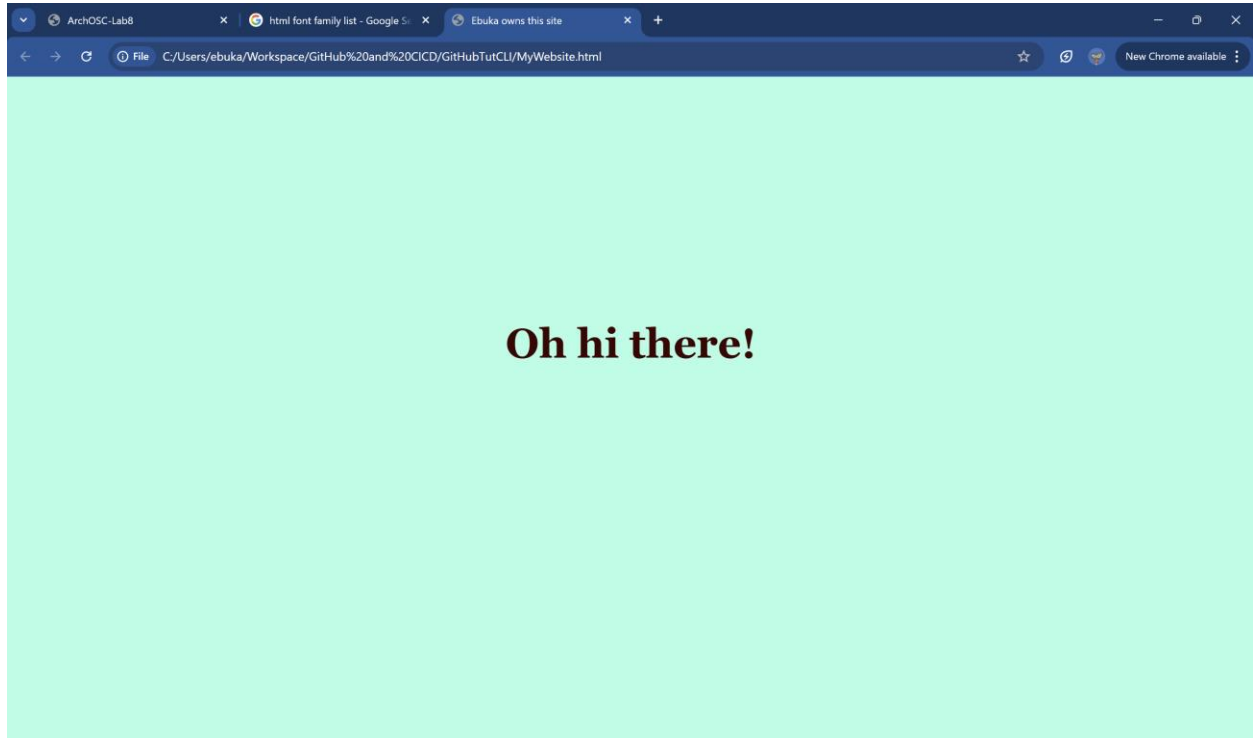
1.7 A screenshot of the completed website

The above shows a website created by copying the given code into visual studio code. This website was saved in our initially created workspace.



1.8 Screenshot of Updated repository including website created

I committed the changes to the linked repository from GitHub desktop and published the changes.

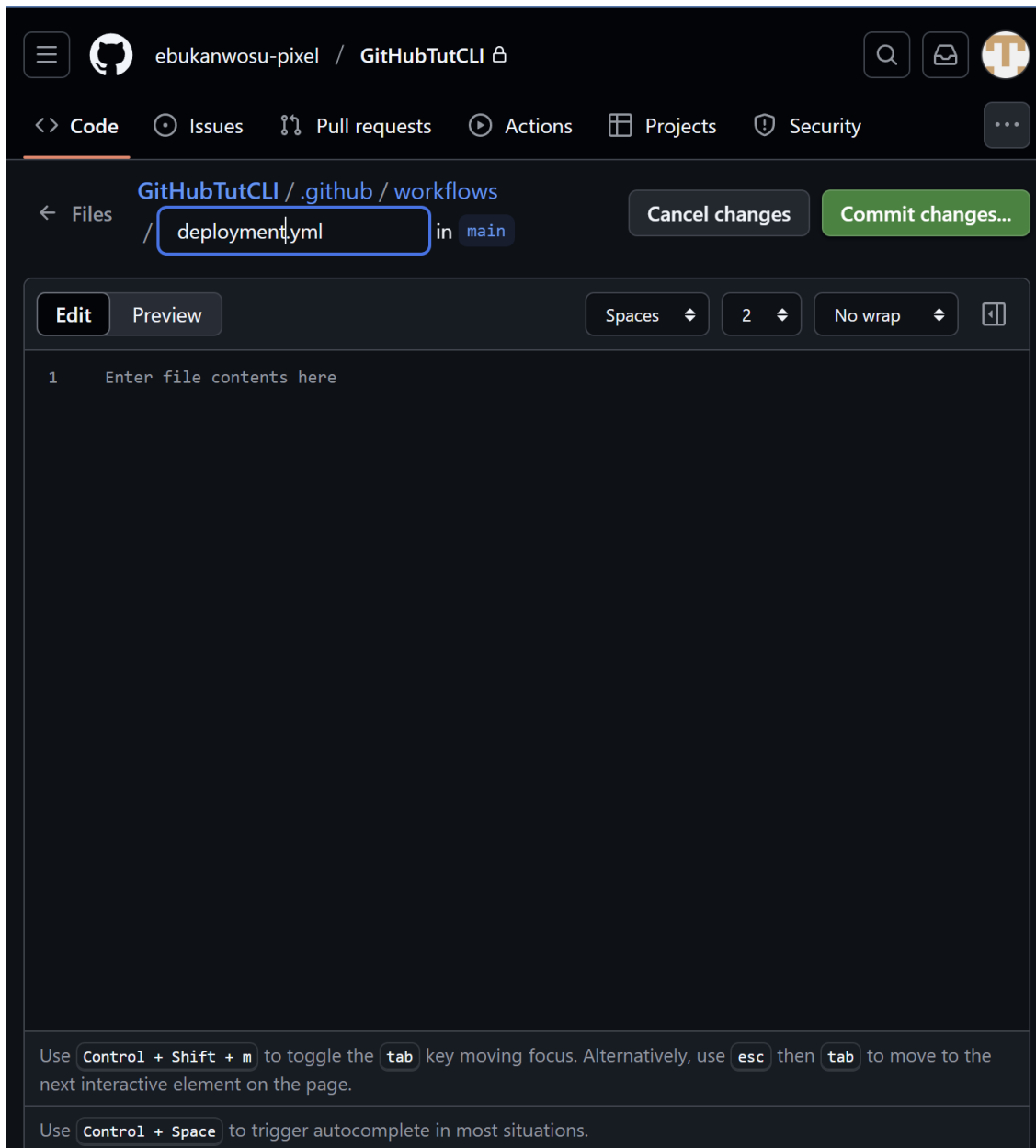


1.9 Updated website

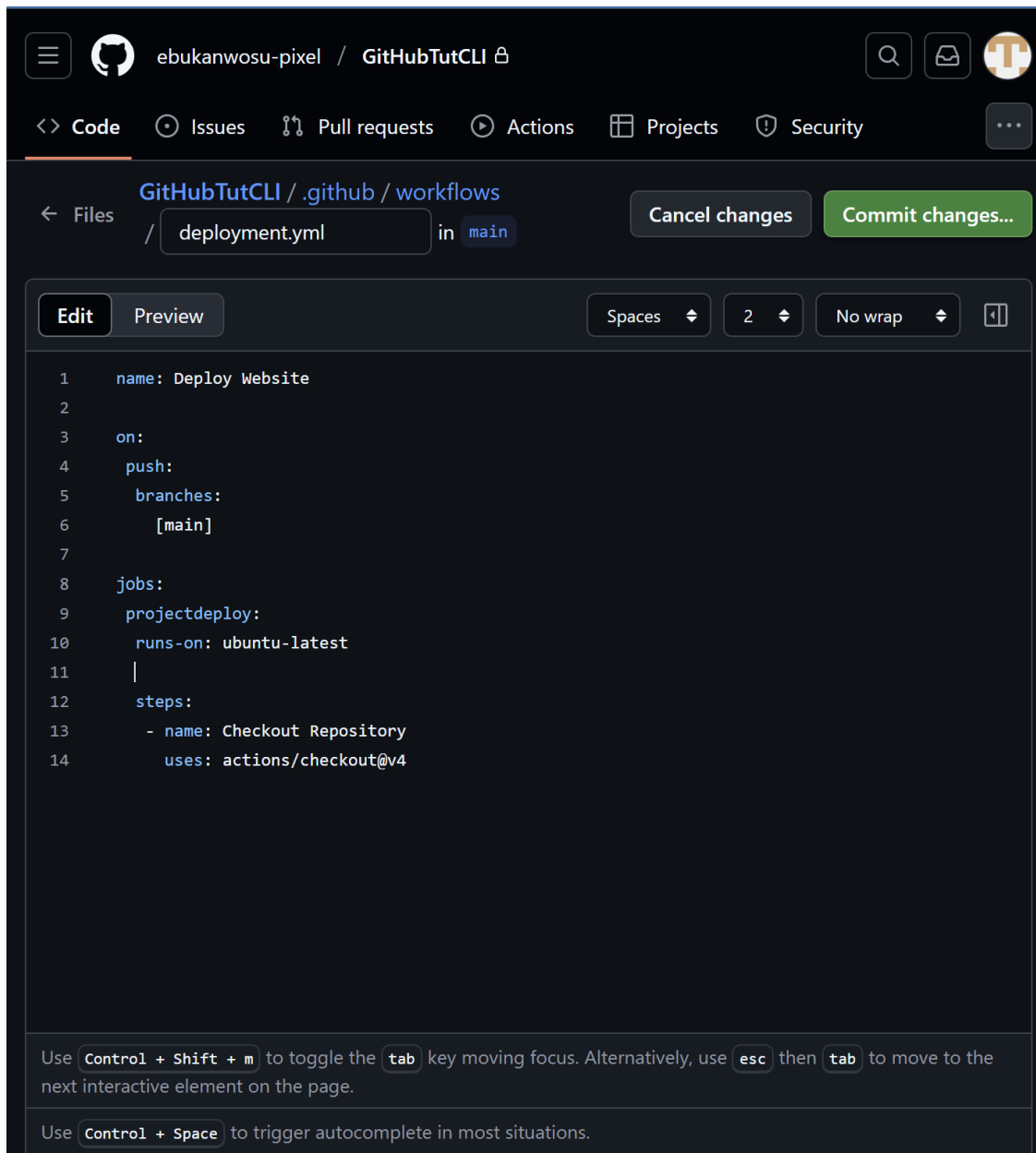
I changed the background colour, font style, font colour and title by altering the code in visual studio code.

*Unable to use github pages on pc without subscribing.

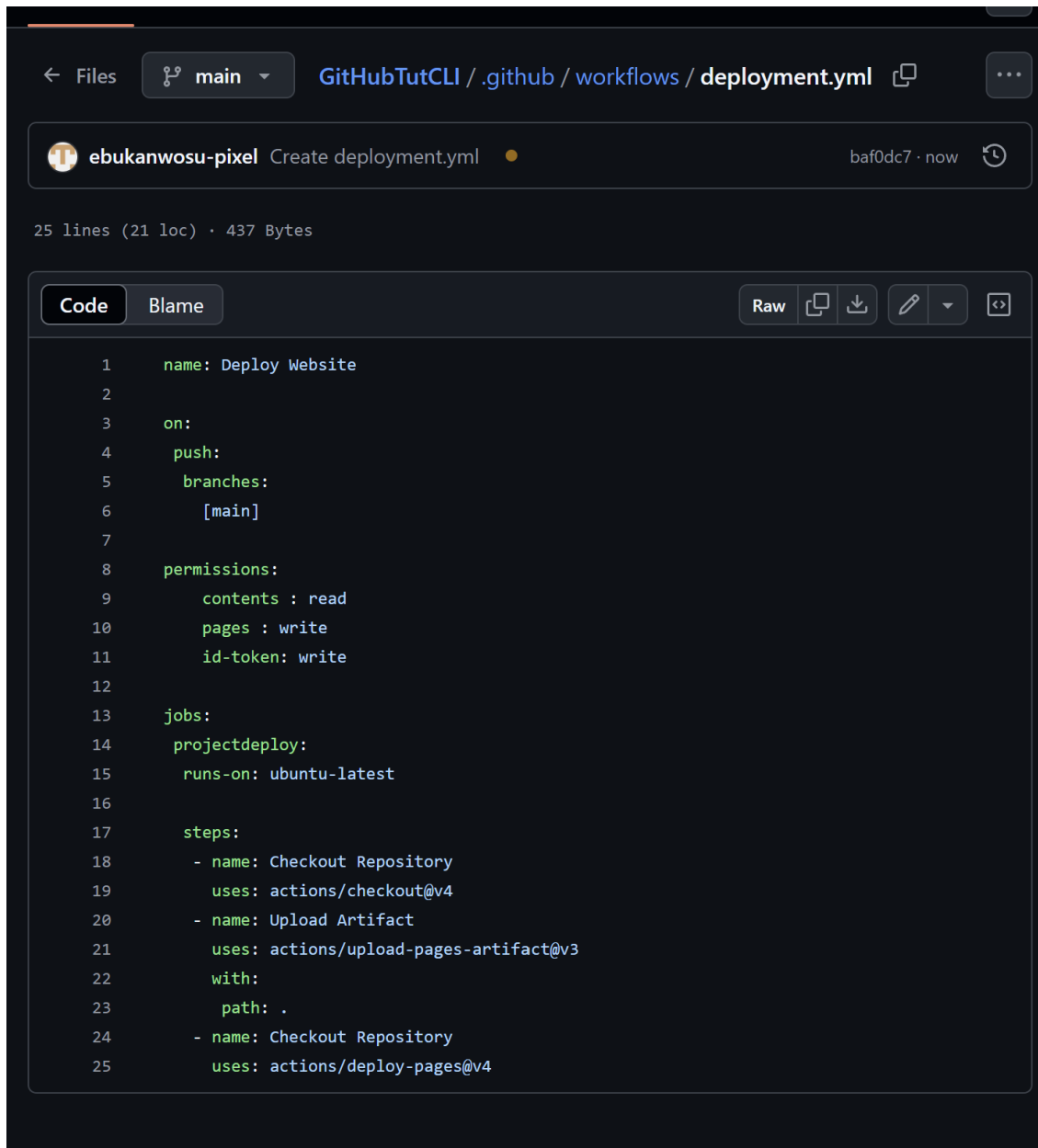
GitHub and CI/CD Lab 2



2.1 Workflow set up and name changed



2.2 Screenshot showing the current level of the YAML that has been put together

A screenshot of a GitHub web interface showing a workflow file named `deployment.yml` in the `.github/workflows` directory. The file is 25 lines long, 21 lines of code, and 437 bytes. The user `ebukanwosu-pixel` created it. The code is a GitHub Actions workflow for deploying a website. It includes a `name`, `on` trigger for pushes to the `main` branch, `permissions` for reading contents, writing pages, and writing id-tokens, and a `jobs` section with a `projectdeploy` job running on `ubuntu-latest`. The job has two steps: one to checkout the repository and upload artifacts, and another to checkout the repository and deploy to GitHub Pages.

```
1  name: Deploy Website
2
3  on:
4    push:
5      branches:
6        [main]
7
8  permissions:
9    contents : read
10   pages : write
11   id-token: write
12
13  jobs:
14    projectdeploy:
15      runs-on: ubuntu-latest
16
17      steps:
18        - name: Checkout Repository
19          uses: actions/checkout@v4
20        - name: Upload Artifact
21          uses: actions/upload-pages-artifact@v3
22          with:
23            path: .
24        - name: Checkout Repository
25          uses: actions/deploy-pages@v4
```

2.3 This is a screenshot showing the completed YAML

The YAML shown above is to deploy the website to GitHub pages so you can access your website even from a mobile device

*GitHub pages does not work on my PC without subscription